

OWL, POINT OF SALE, AND INTEGRATIONS

OWL, POS & External APIs

Frontend components, POS tweaks, and XML-RPC

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026

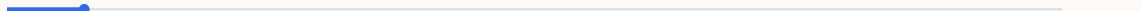
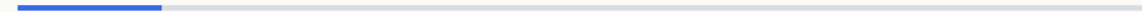


Agenda

- 1 Section 1: OWL Component Basics — 12 topics
- 2 Section 2: POS Customization — 8 topics
- 3 Section 3: XML-RPC and Integration APIs — 13 topics

1 Section 1

OWL Component Basics



1.1 OWL Intro

- OWL is Odoo's JavaScript component framework.

Component idea

Component class + XML template + optional services

- Components combine state, behavior, and templates.

Small import

```
import { Component } from "@odoo/owl";
```

1.2 Component Structure

- A component class owns behavior.

Component class

```
/** @odoo-module **/  
import { Component } from "@odoo/owl";  
  
# ...
```

- A template owns the displayed markup.

Template name

```
<t t-name="training.TrainingCard">  
  <div class="o_training_card">Training</div>  
</t>
```

1.3 setup Method

- setup runs when the component is created.

Use a service

```
setup() {  
    this.notification = useService("notification");  
}
```

- Use it to initialize hooks and services.

Prepare local state

```
setup() {  
    this.state = useState({ ready: false });  
}
```

1.4 useState

- useState creates reactive local state.

Counter state

```
this.state = useState({ count: 0 });  
this.state.count += 1;
```

- Changing state causes the component to rerender.

Display state

```
<span t-out="state.count"/>
```

1.5 Props

- Props pass data from a parent to a child component.

Parent passes prop

```
<TrainingBadge label="'Ready'"/>
```

- The child should treat props as input.

Child reads prop

```
<span t-out="props.label"/>
```


1.6 t-if

- t-if shows markup only when a condition is true.

[Show message](#)

```
<div t-if="state.ready">Ready</div>
```

- Use it for simple visible or hidden blocks.

[Show warning](#)

```
<span t-if="props.total === 0">No lines</span>
```

1.7 t-foreach

- t-foreach repeats markup for a list.

Loop records

```
<li t-foreach="state.records" t-as="record" t-key="record.id">  
  <span t-out="record.name"/>  
</li>
```

- Always provide a stable t-key.

Loop simple values

```
<span t-foreach="props.tags" t-as="tag" t-key="tag">  
  <t t-out="tag"/>  
</span>
```

1.8 t-model

- t-model binds an input to component state.

Text input

```
<input t-model="state.name" placeholder="Course name"/>
```

- It is useful for simple forms.

Checkbox input

```
<input type="checkbox" t-model="state.active"/>
```

1.9 Custom Field Widget Idea

- A field widget customizes how one field appears.

Registry entry

```
registry.category("fields").add("training_badge", {  
    component: TrainingBadgeField,  
});
```

- Register it in the field registry.

Use in XML view

```
<field name="state" widget="training_badge"/>
```

1.10 patch Function

- patch extends existing Odoo JavaScript behavior.

Patch prototype

```
import { patch } from "@web/core/utils/patch";

patch(SomeComponent.prototype, {
  # ...
})
```

- Keep patches narrow and named by purpose.

Call original method

```
patch(SomeComponent.prototype, {
  setup() {
    super.setup(...arguments);
  }
  # ...
})
```

1.11 RPC Service

- The rpc service calls server routes from JavaScript.

Get rpc service

```
setup() {  
  this.rpc = useService("rpc");  
}
```

- Use it at workflow edges, such as save or refresh.

Call a route

```
const result = await this.rpc("/training/course/count", {  
  teacher_id: this.props.teacherId,  
});
```

1.12 Notifications

- Notifications give immediate feedback to users.

Success notification

```
this.notification.add("Course saved", {  
    type: "success",  
});
```

- Use success messages for completed actions.

Warning notification

```
this.notification.add("Select a customer first", {  
    type: "warning",  
});
```

2 Section 2

POS Customization

2.1 POS Custom Button

- A POS button adds a cashier action to the screen.

Button template

```
<t t-name="training.PosButton">  
  <button class="control-button">Training</button>  
</t>
```

- Register it through the POS component system.

Button class

```
export class TrainingPosButton extends Component {  
  static template = "training.PosButton";  
}
```

2.2 POS Click Handler

- A click handler reacts to cashier input.

Template click

```
<button t-on-click="onClick">Training</button>
```

- Keep the handler small and easy to test.

Handler method

```
onClick() {  
    this.notification.add("Button clicked");  
}
```

2.3 Hide POS Button

- Hide a button when the action is not useful.

Template condition

```
<button t-if="shouldShowButton" t-on-click="onClick">  
  Training  
</button>
```

- Use a condition that matches the cashier workflow.

Getter condition

```
get shouldShowButton() {  
  return this.pos.get_order()?.get_orderlines().length > 0;  
}
```

2.4 POS Popup

- A popup asks for confirmation or extra input.

Confirmation call

```
await this.dialog.add(ConfirmationDialog, {  
  title: "Clear order",  
  body: "Remove all order lines?",  
  # ...
```

- Use it when the cashier must make a clear choice.

Popup button

```
<button t-on-click="confirm">Confirm</button>  
<button t-on-click="cancel">Cancel</button>
```

2.5 Translate JS String

- Use `_t` for text shown to users.

Import `_t`

```
import { _t } from "@web/core/l10n/translation";
```

- Translated strings make POS customizations production ready.

Translated label

```
this.notification.add(_t("Order cleared"), {  
    type: "success",  
});
```

2.6 POS RPC

- POS can call the server for live data.

Server call

```
const loyalty = await this.rpc("/pos/loyalty/balance", {  
  partner_id: partner.id,  
});
```

- Keep calls fast because cashiers are waiting.

Route response

```
return {  
  "points": partner.loyalty_points,  
  "name": partner.name,  
  # ...
```

2.7 Clear Order Lines

- Clearing an order removes all current order lines.

Get current order

```
const order = this.pos.get_order();  
const lines = [...order.get_orderlines()];
```

- Ask for confirmation because the action is destructive.

Remove lines

```
for (const line of lines) {  
    order.removeOrderline(line);  
}
```

2.8 POS Assets

- POS JavaScript and XML must be loaded in POS assets.

Manifest assets

```
"point_of_sale._assets_pos": [  
    "training/static/src/pos/**/*",  
]
```

- Add only the files the POS really needs.

XML template file

```
<templates xml:space="preserve">  
    <t t-name="training.PosButton"/>  
</templates>
```


3 Section 3

XML-RPC and Integration APIs

3.1 XML-RPC Auth

- XML-RPC clients authenticate before calling models.

Python authenticate

```
uid = common.authenticate(db, username, password, {})
```

- The result is a user id for later calls.

Connection objects

```
common = xmlrpc.client.ServerProxy(url + "/xmlrpc/2/common")  
models = xmlrpc.client.ServerProxy(url + "/xmlrpc/2/object")
```

3.2 XML-RPC search_read

- search_read finds records and returns selected fields.

Read partners

```
records = models.execute_kw(  
    db, uid, password, "res.partner", "search_read",  
    [[{"is_company", "=", True}]],  
    # ...
```

- It is ideal for simple read integrations.

Read courses

```
courses = models.execute_kw(  
    db, uid, password, "training.course", "search_read",  
    [[{"state", "=", "confirmed"}]],  
    # ...
```

3.3 XML-RPC create

- create inserts a new record through the ORM.

Create partner

```
partner_id = models.execute_kw(  
    db, uid, password, "res.partner", "create",  
    [{"name": "API Customer", "email": "api@example.com"}],  
    # ...
```

- Required fields must be present.

Create course

```
course_id = models.execute_kw(  
    db, uid, password, "training.course", "create",  
    [{"name": "Beginner Odoo"}],  
    # ...
```

3.4 XML-RPC write

- write updates existing records.

Update email

```
ok = models.execute_kw(  
    db, uid, password, "res.partner", "write",  
    [[partner_id], {"email": "new@example.com"}],  
    # ...
```

- Pass a list of ids and a values dictionary.

Archive record

```
models.execute_kw(  
    db, uid, password, "training.course", "write",  
    [[course_id], {"active": False}],  
    # ...
```

3.5 XML-RPC unlink

- unlink deletes records when access rights allow it.

Delete one record

```
models.execute_kw(  
    db, uid, password, "training.course", "unlink",  
    [[course_id]],  
    # ...
```

- Prefer archiving business records when history matters.

Archive instead

```
models.execute_kw(  
    db, uid, password, "training.course", "write",  
    [[course_id], {"active": False}],  
    # ...
```

3.6 Postman Auth Idea

- Postman can test authentication and model calls.

Variables

```
url = https://odoo.example.com  
db = prod_db  
username = api@example.com
```

- Keep database, username, and password in variables.

Auth body idea

```
{  
  "service": "common",  
  "method": "authenticate",  
  # ...
```

3.7 Integration User Least Privilege

- An integration user should have only needed access.

Dedicated login

```
api.training@example.com
```

- Use groups and record rules like any other user.

Limited group

```
<field name="groups_id"  
  eval="[(4, ref('training.group_training_api'))]" />
```


3.8 API Domains

- Domains filter records in API calls.

Active customers

```
[["customer_rank", ">", 0], ["active", "=", True]]
```

- They use the same basic syntax as Odoo search views.

Confirmed courses

```
[["state", "=", "confirmed"], ["start_date", ">=", "2026-01-01"]]
```

3.9 API Field Selection

- Requesting fewer fields makes API calls faster.

Partner fields

```
{"fields": ["name", "email", "phone"]}
```

- It also avoids exposing data the integration does not need.

Course fields

```
{"fields": ["name", "state", "teacher_id"]}
```

3.10 API Error Handling

- APIs can fail because of access, validation, or network issues.

Try and log

```
try:  
    call_odoo()  
except Exception as exc:  
    # ...
```

- Log enough context to retry safely.

Check result

```
if not ok:  
    raise RuntimeError("Odoo write did not succeed")
```

3.11 API Pagination

- Large reads should use limit and offset.

Batch options

```
{"limit": 100, "offset": 0, "order": "id"}
```

- Process records in batches.

Next batch

```
offset += limit  
records = search_read(domain, fields, offset=offset)
```

3.12 API Create vs Update

- Decide how the external system identifies a record.

Find by reference

```
domain = [{"x_external_ref", "=", external_ref}]
```

- Search first when updating an existing business object.

Create or write idea

```
if record_id:  
    write(record_id, values)  
else:  
    # ...
```

3.13 API Test Data Cleanup

- Test integrations with safe sample records.

Test marker

```
{"name": "API TEST Customer", "x_test_record": True}
```

- Mark test data clearly.

Cleanup domain

```
[["x_test_record", "=", True], ["create_uid", "=", uid]]
```

NEXT STEP

Try the examples on your machine

Then open the next module deck

Merit Advisory

06 · OWL / POS / API

